

RAG Copilot Week 4 专属执行手册

只服务 enterprise-rag-copilot 的最终包装、复现、演示、面试与投递资产

Chenning + ChatGPT

生成日期：2026-06-10

文档属性	说明
适用项目	enterprise-rag-copilot：可迁移企业级 RAG Copilot，电商售后客服为首个 domain demo。
Week 4 定位	RAG 不再扩散功能，而是把 Week 1 的 naive RAG v0 与 Week 2 的 SQL + 文档混合问答、evidence、fallback、evaluation、performance tracing 整理成可展示、可复现、可写进简历的项目版本。
执行对象	Chenning 人工 owner + Codex 候选实现生成者。Codex 只能按任务卡改小范围文件，不能自由重构项目。
最高约束	完全遵守 Vibe Coding 项目实践公约：AI 输出是候选稿；投递资产必须基于真实代码、文档、trace 或 benchmark；任何指标不得虚构。
本周核心交付	README、architecture、api、evaluation、performance、failure_cases、demo_script、interview_notes、sample traces、benchmarks、resume bullets。

核心判断：Week 4 的 RAG 不是继续加高级功能，而是把已经做出来的系统变成可交付资产：别人能跑、你能讲、指标可复现、失败可解释、简历可追问。

0. 静态目录

- 1. 文档定位与 Week 4 RAG 核心判断
- 2. 实践公约在 Week 4 RAG 包装中的落地方式
- 3. Week 4 RAG 总交付物与推荐补齐目录
- 4. Codex 工作协议：投递资产生成规则
- 5. Day 1 - Packaging Baseline：检查 Week 1-3 产物并建立包装 TODO
- 6. Day 2 - Failure Cases + Traces：补失败样例和三类 sample trace
- 7. Day 3 - API Contract：补 docs/api.md 与 evidence schema 说明
- 8. Day 4 - Evaluation Report：补 evaluation.md、eval_report 与 30 条 case 检查
- 9. Day 5 - Performance Report：补 performance.md、benchmark 与 cache 对比
- 10. Day 6 - README + Demo + Resume：完成展示资产和简历 bullet
- 11. Day 7 - Final QA + Career Handoff：最终验收、讲稿、投递材料
- 12. 最终验收清单、面试表达、附录模板

1. 文档定位与 Week 4 RAG 核心判断

Week 4 的原始主题是 RepoPilot 工程化 + 两个项目包装 + 简历投递准备。对 RAG Copilot 来说，本周的主线不是继续做新能力，而是完成最终整理和包装：README 能让别人启动，architecture 能讲清 core pipeline + domain adapter，API 文档能说明请求/响应，evaluation 能说明质量如何评估，performance 能说明指标来自哪里，failure_cases 能说明系统失败时如何处理，demo script 能支撑 3-5 分钟演示，简历 bullet 能被代码和实验支撑。因此，本 PDF 把 Week 4 中所有 RAG 相关内容单独抽出，并改写成 Codex 可以执行、Chenning 可以 review 的每日任务书。

维度	Week 4 RAG 应该做	Week 4 RAG 不应该做
功能方向	整理 Week 1-2 已有能力：SQL tool、document retrieval、intent router、evidence、fallback、evaluation、performance tracing。	继续堆 GraphRAG、多模态、复杂 reranker、生产级缓存或大型监控系统。
文档方向	补齐 README、architecture、api、evaluation、performance、failure_cases、demo_script、interview_notes。	只写口号式 README，不给启动命令、API 示例、demo query 和已知局限。
指标方向	只保留由本地脚本、日志或 benchmark 产生的 latency、error rate、cache comparison 等数字。	为了简历好看虚构 QPS、P95/P99、cache hit rate 或生产级结论。
面试方向	准备 3 分钟项目讲稿和至少 20 个 Q&A，能解释设计取舍与局限。	把项目包装成生产级系统，或说不清每个 bullet 的代码依据。

2. 实践公约在 Week 4 RAG 包装中的落地方式

Week 4 是最接近生产区的一周，因为产物会进入 GitHub、简历、面试讲稿和投递材料。这里的“生产区”不是指真实线上生产系统，而是指项目资产会被别人阅读、复现、追问。

任务类型	区域判断	Codex 可以做	Chenning 必须 Review
README / architecture / API 文	生产区文档	生成草稿、补结构、补示例、统	启动命令是否真的能跑；架构是

档		一术语。	否准确；是否夸大能力。
evaluation / failure_cases	生产区评估资产	整理 cases、补报告模板、补失败分类。	case 是否真实；失败原因是否可解释；是否只记录成功案例。
performance / benchmarks	生产区指标资产	整理脚本输出、写实验环境、生成报告模板。	数字是否来自可复现实验；样本量与局限是否写清。
sample traces	生产区证据资产	根据真实运行日志整理 trace JSON。	trace 是否能显示 route/retrieval/tool/generation/fallback；是否泄露密钥或隐私。
resume bullets / interview notes	投递资产	生成候选 bullet 和 Q&A 草稿。	每条 bullet 是否有代码、文档、trace 或 benchmark 支撑；是否会被追问穿帮。

2.1 Week 4 RAG 开发硬规则

1. 不再为了“看起来高级”增加 RAG 新功能，发现缺口后优先补关键验收与包装资产。
2. README 不能只写愿景，必须包含启动方式、API 示例、demo query、Week 1-4 能力和已知局限。
3. performance.md 只能写可复现指标；如果没有测完，必须明确写 TODO / 未完成，不能虚构。
4. failure_cases.md 必须记录失败样例和处理方式，不能只展示成功 demo。
5. sample trace 至少覆盖订单/结构化查询、政策/文档查询、fallback 三类链路。
6. 简历 bullet 必须能在代码、文档、trace、benchmark 或 eval case 中找到支撑。
7. Codex 输出必须经过人工 Review Gate：事实性、边界、可复现性、无夸大、无敏感信息。

3. Week 4 RAG 总交付物与推荐补齐目录

```
enterprise-rag-copilot/  
  README.md  
  docs/  
    architecture.md  
    api.md  
    evaluation.md  
    performance.md  
    failure_cases.md  
    demo_script.md  
    interview_notes.md  
  eval/  
    ecommerce_cases.json  
    eval_report.json  
  traces/  
    sample_order_query_trace.json  
    sample_policy_query_trace.json  
    sample_fallback_trace.json  
  benchmarks/  
    local_latency_report.md  
    cache_comparison.md  
  career_materials/  
    resume_project_bullets.md  
    rag_project_pitch.md  
    interview_qa_rag.md  
  docker-compose.yml # optional; if incomplete, document current local startup instead
```

RAG Copilot Week 4 专属执行手册 | final packaging, reproducible evidence, no fake metrics

交付物	最低内容	验收方式
README.md	背景、架构、快速启动、API 示例、demo query、evaluation、performance、failure cases、known limitations。	陌生人按 README 能启动或明确知道当前启动方式。
docs/api.md	/health、/chat、/retrieve 如已有；request/response 示例；Evidence / Fallback / Trace 字段解释。	curl 示例能对应真实 schema。
docs/evaluation.md	intent accuracy、retrieval hit、citation correctness、answer faithfulness、fallback correctness。	能说明评价方法和 limitations。
docs/performance.md	环境、接口、样本量、命令、avg latency、P95/P99 如已测、error rate、cache 对比。	每个数字有来源；未测项明确写未完成。
docs/failure_cases.md	检索失败、缺订单号、低置信度、政策冲突、证据不足等。	至少 5 类失败/局限，包含处理方式。
traces/*.json	order query、policy query、fallback query 三类 sample trace。	每个 trace 能显示 route、tool/retrieval、evidence/fallback、latency。
career materials	3-5 条简历 bullet、3 分钟讲稿、20 个 Q&A。	每条表达能回指到代码/文档/指标。

4. Codex 工作协议：投递资产生成规则

给 Codex 的总提示： 你正在维护 enterprise-rag-copilot 的 Week 4 包装资产。你的输出是候选稿，不是最终稿。每次只改本任务指定文件，不允许重写 core pipeline，不允许虚构功能或指标。所有指标必须来自已有日志、脚本输出或明确标记为 TODO。

字段	要求
Context	项目是可迁移企业级 RAG Copilot，电商售后客服只是首个 domain demo。Week 4 目标是包装、复现、演示、面试和投递资产。
Constraints	禁止新增大功能；禁止虚构指标；禁止把未实现内容写成已完成；禁止删除 failure case；禁止暴露 API key / 隐私数据。
Process	先列出将修改的文件，再生成候选内容，再给验收命令或人工检查项，最后给 Review Checklist。
Review Gate	Chenning 必须确认：内容真实、可复现、不夸大、符合架构边界、能支撑简历追问。

5. Day 1 - Packaging Baseline：检查 Week 1-3 产物并建立包装 TODO

目标： 先确认 RAG 现有能力和文档缺口，不直接写包装文案。今天的核心产出是 docs/week4_packaging_todo.md。

项目项	内容
允许修改文件	docs/week4_packaging_todo.md README.md docs/architecture.md docs/evaluation.md docs/performance.md docs/failure_cases.md
给 Codex 的任务卡	扫描仓库中 README、docs、eval、traces、benchmarks、scripts、pipeline、domains/ecommerce 的现状，生成 Week 4

	RAG packaging TODO。不要修改 core pipeline。
建议 commit	docs(rag): add week4 packaging todo and baseline gaps

5.1 今日执行步骤

- 1. 检查 /chat 是否仍能演示混合流程；若不能，记录为 high-priority gap。
- 2. 检查 README 是否包含 quick start、API 示例、demo query、known limitations。
- 3. 检查 docs/architecture.md 是否清楚表达 core pipeline + ecommerce domain adapter。
- 4. 检查 docs/evaluation.md / performance.md / failure_cases.md 是否存在空泛或虚构内容。
- 5. 列出缺失文件：api.md、demo_script.md、interview_notes.md、sample traces、benchmarks、resume bullets。

5.2 验收命令 / 手动检查

```
python -m uvicorn app.main:app --reload
curl http://localhost:8000/health
curl -X POST http://localhost:8000/chat -H "Content-Type: application/json" -d '{"query":"我的订单 ORDER_001 到哪里了? '"
```

5.3 Review Gate

- TODO 必须按 Must / Should / Optional 分级。
- 不能把未实现能力写成已完成。
- 发现指标缺失时写 TODO，不编造。

5.4 今日失败记录模板

```
# Day 1 RAG Failure / Packaging Log
- Date:
- Task:
- File(s) changed:
- What failed or was missing:
- Evidence / command / screenshot:
- Root cause hypothesis:
- Fix or documentation update:
- Review decision: Accept / Must Fix / Follow-up
- Commit: docs(rag): add week4 packaging todo and baseline gaps
```

6. Day 2 - Failure Cases + Traces：补失败样例和三类 sample trace

目标：把 Week 2/3 失败案例整理成可展示资产，并准备三类 trace：订单查询、政策查询、fallback 查询。

项目项	内容
允许修改文件	docs/failure_cases.md traces/sample_order_query_trace.json traces/sample_policy_query_trace.json traces/sample_fallback_trace.json
给 Codex 的任务卡	根据已有 evaluation/failure logs 整理 failure_cases.md 和 sample trace 模板。只能使用真实字段或 mock trace 模板；不得伪造真实性数据。
建议 commit	docs(rag): add failure cases and sample traces

6.1 今日执行步骤

1. failure_cases.md 至少包含：缺少订单号、检索低分、证据不足、政策冲突、unknown intent。
2. 为每个 failure case 写：user query、expected behavior、actual behavior、root cause、fallback behavior、status。
3. 整理 sample_order_query_trace.json：包含 route、structured tool、evidence、latency 字段。
4. 整理 sample_policy_query_trace.json：包含 route、retrieval、top chunks、evidence 字段。
5. 整理 sample_fallback_trace.json：包含 fallback_reason 和 user-facing response。

6.2 验收命令 / 手动检查

```
python evaluation/run_eval.py --suite evaluation/ecommerce_cases.json
python scripts/run_demo_queries.py --save-traces traces/ # 若脚本存在
python -m json.tool traces/sample_fallback_trace.json
```

6.3 Review Gate

- trace 不应包含 API key、真实用户隐私或不可公开日志。
- failure case 不能只写“失败了”，必须写处理方式。
- sample trace 若为 mock，需要在文件顶部或 README 中标记。

6.4 今日失败记录模板

```
# Day 2 RAG Failure / Packaging Log
- Date:
- Task:
- File(s) changed:
- What failed or was missing:
- Evidence / command / screenshot:
- Root cause hypothesis:
- Fix or documentation update:
- Review decision: Accept / Must Fix / Follow-up
- Commit: docs(rag): add failure cases and sample traces
```

7. Day 3 - API Contract: 补 docs/api.md 与 evidence schema 说明

目标：让 /chat 的输入输出契约清楚可读，尤其是 answer、intent、evidence、fallback、trace_id。

项目项	内容
允许修改文件	docs/api.md schemas/chat.py schemas/evidence.py README.md
给 Codex 的任务卡	根据现有 Pydantic schema 和路由实现生成 docs/api.md。不要改接口实现，除非发现文档与代码不一致时先报告。
建议 commit	docs(rag): document chat api evidence and fallback contract

7.1 今日执行步骤

1. 写 /health：用途、method、response 示例。
2. 写 /chat：request fields、response fields、成功示例、fallback 示例。
3. 如已有 /retrieve，写 retrieve 示例；没有则明确写未提供 standalone retrieve API。
4. 解释 Evidence 字段：source、content、score、evidence_type、metadata 等。
5. 解释 fallback 字段：fallback、fallback_reason、下一步建议。

7.2 验收命令 / 手动检查

```
curl http://localhost:8000/health  
curl -X POST http://localhost:8000/chat -H "Content-Type: application/json" -d '{"query":"退货政策是什么? "'  
pytest tests/ -q # 若已有测试
```

7.3 Review Gate

- API 示例必须和真实字段一致。
- 如果代码里没有某字段，不能在文档里写成已有。
- 业务回答示例必须带 evidence 或 fallback。

7.4 今日失败记录模板

```
# Day 3 RAG Failure / Packaging Log  
- Date:  
- Task:  
- File(s) changed:  
- What failed or was missing:  
- Evidence / command / screenshot:  
- Root cause hypothesis:  
- Fix or documentation update:  
- Review decision: Accept / Must Fix / Follow-up  
- Commit: docs(rag): document chat api evidence and fallback contract
```

8. Day 4 - Evaluation Report: 补 evaluation.md、eval_report 与 30 条 case 检查

目标：把“效果看起来还行”变成可解释的评估方法：intent、retrieval、citation、faithfulness、fallback。

项目项	内容
允许修改文件	docs/evaluation.md eval/ecommerce_cases.json eval/eval_report.json evaluation/run_eval.py evaluation/metrics.py
给 Codex 的任务卡	检查 evaluation cases 是否至少 30 条，按结构化问题、文档问题、混合问题、失败问题分类。生成 evaluation.md 和 eval_report 模板。
建议 commit	docs(rag): finalize evaluation method and report

8.1 今日执行步骤

- 统计 cases 数量与类型覆盖：structured / document / hybrid / failure。
- 在 docs/evaluation.md 中解释 intent accuracy、retrieval hit、citation correctness、answer faithfulness、fallback correctness。
- 如已有 run_eval.py，运行并保存 eval_report.json；如没有，写最小可运行脚本或明确 TODO。
- 补充 evaluation limitations：小样本、本地 mock/LLM 模式、无人工标注规模化评估。

8.2 验收命令 / 手动检查

```
python evaluation/run_eval.py --suite evaluation/ecommerce_cases.json --out eval/eval_report.json

python - <<"PY"
import json;print(len(json.load(open("eval/ecommerce_cases.json"))))
PY

python -m json.tool eval/eval_report.json
```

8.3 Review Gate

- 若不足 30 条 case，不能在 README 写“30+”。
- 评估指标定义要对应项目当前能力，不强行套复杂框架。
- 报告必须写 limitations。

8.4 今日失败记录模板

```
# Day 4 RAG Failure / Packaging Log
- Date:
- Task:
- File(s) changed:
- What failed or was missing:
- Evidence / command / screenshot:
- Root cause hypothesis:
- Fix or documentation update:
- Review decision: Accept / Must Fix / Follow-up
- Commit: docs(rag): finalize evaluation method and report
```

9. Day 5 - Performance Report: 补 performance.md、benchmark 与 cache 对比

目标：把性能表达从“我测过”变成“环境、命令、样本、结果、局限都可复现”。

项目项	内容
允许修改文件	docs/performance.md benchmarks/local_latency_report.md benchmarks/cache_comparison.md scripts/load_test.py pipeline/performance_tracer.py pipeline/cache.py
给 Codex 的任务卡	整理已有 performance tracer、load test、cache demo 输出，生成 performance.md 和 benchmarks。若没有真实结果，必须写未完成和下一步命令，不得伪造。
建议 commit	docs(rag): add reproducible performance report

9.1 今日执行步骤

1. 写测试环境：OS、Python、LLM mode、dataset size、API server command。
2. 写测试接口：/health、/chat、/retrieve 如已有。
3. 记录 avg latency、P95/P99、error rate、cache comparison；没有就写 TODO。
4. 说明本地小样本实验不能等同生产指标。
5. 把 cache demo 的收益和风险写清楚：FAQ 可以缓存，订单类数据需谨慎。

9.2 验收命令 / 手动检查

```
python scripts/load_test.py --endpoint http://localhost:8000/chat --n 30
python scripts/load_test.py --endpoint http://localhost:8000/health --n 50
python scripts/cache_smoke_test.py # 若存在
```

9.3 Review Gate

- 数字必须来自命令输出或日志。
- 没有测 P95/P99 就写未完成，不能估计。
- cache hit rate 若无脚本统计，不写具体百分比。

9.4 今日失败记录模板

```
# Day 5 RAG Failure / Packaging Log
- Date:
- Task:
- File(s) changed:
- What failed or was missing:
- Evidence / command / screenshot:
- Root cause hypothesis:
- Fix or documentation update:
- Review decision: Accept / Must Fix / Follow-up
- Commit: docs(rag): add reproducible performance report
```

10. Day 6 - README + Demo + Resume：完成展示资产和简历 bullet

目标：把 RAG 项目变成别人能看懂、能跑、你能演示、能投递的项目资产。

项目项	内容
允许修改文件	README.md docs/demo_script.md docs/interview_notes.md career_materials/resume_project_bullets.md docker-compose.yml
给 Codex 的任务卡	基于真实代码和文档生成 README、demo script、interview notes 和简历 bullet 候选稿。所有 bullet 必须有支撑来源，不得夸大。
建议 commit	docs(rag): finalize readme demo and resume bullets

10.1 今日执行步骤

1. README 补齐：Project Motivation、Architecture、Quick Start、API Examples、Demo Queries、Evaluation、Performance、Failure Cases、Limitations。
2. docs/demo_script.md 写 3-5 分钟演示流程：order query、policy query、hybrid query、fallback query。
3. docs/interview_notes.md 至少 20 个 RAG Q&A。
4. 写 3-5 条简历 bullet：技术方案、工程化、评估/性能、个人贡献。
5. Docker Compose 如未完整实现，README 写清当前本地启动方式和 compose TODO。

10.2 验收命令 / 手动检查

```
python -m uvicorn app.main:app --reload
curl -X POST http://localhost:8000/chat -H "Content-Type: application/json" -d '{"query":"ORDER_001 可以退货吗? "'
docker compose up --build # 若已支持
```

10.3 Review Gate

- README 中每个 demo query 都应实际跑过。
- 简历 bullet 不写“生产级”“大规模”“高并发”除非有真实证据。
- interview notes 要包含局限和 trade-off。

10.4 今日失败记录模板

```
# Day 6 RAG Failure / Packaging Log
- Date:
- Task:
- File(s) changed:
- What failed or was missing:
- Evidence / command / screenshot:
- Root cause hypothesis:
- Fix or documentation update:
- Review decision: Accept / Must Fix / Follow-up
- Commit: docs(rag): finalize readme demo and resume bullets
```

11. Day 7 - Final QA + Career Handoff: 最终验收、讲稿、投递材料

目标：完成 RAG 项目的最终验收，并把输出移交到 career_materials，准备投递与面试。

项目项	内容
允许修改文件	career_materials/rag_project_pitch.md career_materials/interview_qa_rag.md career_materials/demo_checklist.md career_materials/application_todo.md README.md
给 Codex 的任务卡	根据当前 README/docs/traces/benchmarks 生成 RAG 3 分钟讲稿、Q&A 汇总、demo checklist 和投递 TODO。不要新增功能。
建议 commit	chore(rag): final qa and career handoff materials

11.1 今日执行步骤

1. 按最终验收清单检查 RAG 是否可启动、/chat 是否可演示、evidence/fallback/eval/performance/docs 是否齐全。
2. 写 RAG 3 分钟讲稿：背景、架构、核心功能、工程化、结果与局限。
3. 整理 interview_qa_rag.md：至少 20 个 Q&A。
4. 整理 demo_checklist.md：演示命令、demo query、预期输出、失败备用方案。
5. 整理 application_todo.md：简历替换、GitHub README 检查、投递岗位匹配。

11.2 验收命令 / 手动检查

```
python -m uvicorn app.main:app --reload
python evaluation/run_eval.py --suite evaluation/ecommerce_cases.json
python scripts/load_test.py --endpoint http://localhost:8000/chat --n 10
```

11.3 Review Gate

- 讲稿必须诚实说明“当前是可投递雏形，不是生产级系统”。
- 投递材料中每个数字必须能回到 benchmark。
- demo checklist 要有 fallback plan。

11.4 今日失败记录模板

```
# Day 7 RAG Failure / Packaging Log
- Date:
- Task:
- File(s) changed:
- What failed or was missing:
- Evidence / command / screenshot:
- Root cause hypothesis:
- Fix or documentation update:
- Review decision: Accept / Must Fix / Follow-up
- Commit: chore(rag): final qa and career handoff materials
```

12. Week 4 RAG 最终验收清单

类别	验收项
运行与接口	☐ 项目可本地启动 ☐ /health 可访问 ☐ /chat 可演示当前混合流程 ☐ 回答包含 evidence citation ☐ fallback 可演示
文档资产	☐ README 可展示 ☐ docs/architecture.md 已更新 ☐ docs/api.md 已完成 ☐ docs/evaluation.md 已完成 ☐ docs/performance.md 已完成 ☐ docs/failure_cases.md 已完成 ☐ docs/demo_script.md 已完成 ☐ docs/interview_notes.md 至少 20 个 Q&A
评估与指标	☐ eval/ecommerce_cases.json 至少 30 条 ☐ eval/eval_report.json 或等价报告存在 ☐ traces/ 至少 3 个 sample trace ☐ benchmarks/ 至少 1 个本地实验记录 ☐ 所有指标写清来源和局限
投递资产	☐ RAG 3 分钟讲稿 ☐ 简历 bullet 3-5 条 ☐ demo checklist ☐ application todo ☐ 每条 bullet 都有支撑材料

13. RAG 3 分钟讲稿骨架

1. 项目背景：企业知识问答不是简单聊天，需要同时结合结构化业务数据和非结构化政策文档。
2. 架构设计：系统采用 core pipeline + ecommerce domain adapter，核心流程不写死电商规则。
3. 核心功能：intent routing、structured tools、document retrieval、evidence citation、fallback、evaluation、performance tracing。
4. 工程化：FastAPI / Pydantic 固定接口契约，logging / trace 记录链路，evaluation cases 和 benchmarks 支撑质量与性能表达。
5. 结果与局限：当前是可投递、可展示、可迭代的工程雏形；本地指标不等于生产级指标，后续可扩展到 HR/IT support 等 domain。

14. 简历 bullet 候选模板（必须按实际结果替换）

使用规则： 下面是模板，不是可以直接夸大的成品。凡是含数字的位置必须替换为真实实验结果；没有真实数据就删掉或写“构建了可复现评测流程”。

- 设计并实现可迁移企业级 RAG Copilot 框架，将业务逻辑抽象为 Domain Adapter，支持结构化业务查询、非结构化文档检索、证据融合、fallback 和性能追踪。

- 以电商售后为首个 domain demo，接入订单/商品/退款模拟数据与 FAQ/退换货/物流/保修文档，实现订单状态、政策问答和混合问题处理。
- 设计 evidence citation 与 low-confidence fallback 机制，使关键业务回答可追溯到数据库结果或文档片段，并在证据不足时给出转人工建议。
- 构建 evaluation cases 评估 intent accuracy、retrieval hit、citation correctness、answer faithfulness 与 fallback correctness，沉淀 failure_cases.md 用于持续修复。
- 实现本地 performance tracing 与压测记录流程，统计 SQL/tool、retrieval、LLM/mock、total latency、error rate、cache 对比等指标，所有数字来自可复现实验。

15. 附录 A: docs/api.md 最小结构

```
# API Reference - enterprise-rag-copilot

## GET /health
Purpose: check service availability.
Response:
{
  "status": "ok"
}

## POST /chat
Request:
{
  "user_id": "u001",
  "session_id": "s001",
  "query": "ORDER_001 可以退货吗? "
}

Response:
{
  "answer": "...",
  "intent": "hybrid",
  "evidence": [
    {"type": "structured", "source": "orders", "content": "...", "score": 1.0},
    {"type": "document", "source": "return_policy.md", "content": "...", "score": 0.82}
  ],
  "fallback": false,
  "fallback_reason": null,
  "trace_id": "trace_xxx"
}

## Fallback response example
{
  "answer": "我暂时无法确认该订单信息，建议提供订单号或转人工处理。",
  "intent": "unknown",
  "evidence": [],
  "fallback": true,
  "fallback_reason": "missing_order_id",
  "trace_id": "trace_xxx"
}
```

16. 附录 B: docs/performance.md 最小可信格式

```
# Performance Report - enterprise-rag-copilot

## Environment
- Machine:
- Python:
- LLM mode: mock / API
- Dataset size: docs=<n>, chunks=<n>, eval_cases=<n>
- Server command: uvicorn app.main:app --reload

## Commands
- python scripts/load_test.py --endpoint http://localhost:8000/health --n 50
- python scripts/load_test.py --endpoint http://localhost:8000/chat --n 30
- python evaluation/run_eval.py --suite evaluation/ecommerce_cases.json

## Results
| Metric | Value | Source |
|---|---|---|
| avg_total_latency_ms | <measured or TODO> | load_test.py |
| p95_total_latency_ms | <measured or TODO> | load_test.py |
| error_rate | <measured or TODO> | load_test.py |
| cache_comparison | <measured or TODO> | cache_comparison.md |

## Limitations
- Local experiment only, not production deployment.
- Small sample size.
- Mock LLM if applicable.
- Do not present these as production QPS/P95 numbers.
```

17. 附录 C: docs/interview_notes.md 至少 20 个 RAG Q&A

1. 你的 RAG 项目和普通知识库问答有什么区别？
2. 为什么要区分 core pipeline 和 domain adapter？
3. 为什么订单状态不应该靠向量检索？
4. SQL 工具和文档检索分别解决什么问题？
5. evidence 字段有什么作用？
6. fallback 什么时候触发？
7. 你如何避免模型编造答案？
8. evaluation cases 如何设计？
9. intent accuracy 如何评估？
10. retrieval hit 如何评估？
11. citation correctness 如何评估？
12. answer faithfulness 如何评估？
13. performance tracer 记录哪些阶段？
14. cache demo 缓存了什么？

15. 缓存会带来什么风险？
16. P95/P99 和平均延迟有什么区别？
17. 你的性能指标来自哪里？
18. 这个项目为什么可迁移？
19. 如果换成 HR/IT support 场景，需要改哪些模块？
20. 这个项目目前最大的局限是什么？

18. Week 4 RAG 最重要的一句话

Week 4 的 RAG 成功标准不是“继续加了多少功能”，而是：这个项目能被别人启动，能用 3-5 分钟清楚演示，能用 evaluation / traces / benchmarks 支撑你的简历表达，能诚实说明失败和局限，并且每条投递表达都经得起面试追问。